We're slowly building up our skills, and we want our algorithm to train on the harder variant of the game where all the board pieces are randomly placed on the grid for each new game. The algorithm can't just memorize a sequence of steps to take, as before. It needs to be able to take the shortest path to the goal (without stepping into the pit) regardless of what the initial board configuration is. It needs to develop a more sophisticated representation of its environment.

3.3.1. Catastrophic forgetting

The main problem we encountered in the previous section when we tried to train our model on random mode has a name: *catastrophic forgetting*. It's actually a very important issue associated with gradient descent-based training methods in *online* training. Online training is what we've been doing: we backpropagate after each move as we play the game.

Imagine that our algorithm is training on (learning Q values for) game 1 of figure 3.12. The player is placed between the pit and the goal such that the goal is on the right and the pit is on the left. Using an epsilon-greedy strategy, the player takes a random move and by chance steps to the right and hits the goal. Great! The algorithm will try to learn that this state-action pair is associated with a high value by updating its weights in such a way that the output will more closely match the target value (i.e. via backpropagation).

Figure 3.12. The idea of catastrophic forgetting is that when two game states are very similar and yet lead to very different outcomes, the Q function will get "confused" and won't be able to learn what to do. In this example, the catastrophic forgetting happens because the Q function learns from game 1 that moving right leads to a +1 reward, but in game 2, which looks very similar, it gets a reward of –1 after moving right. As a result, the algorithm forgets what it previously learned about game 1, resulting in essentially no significant learning at all.